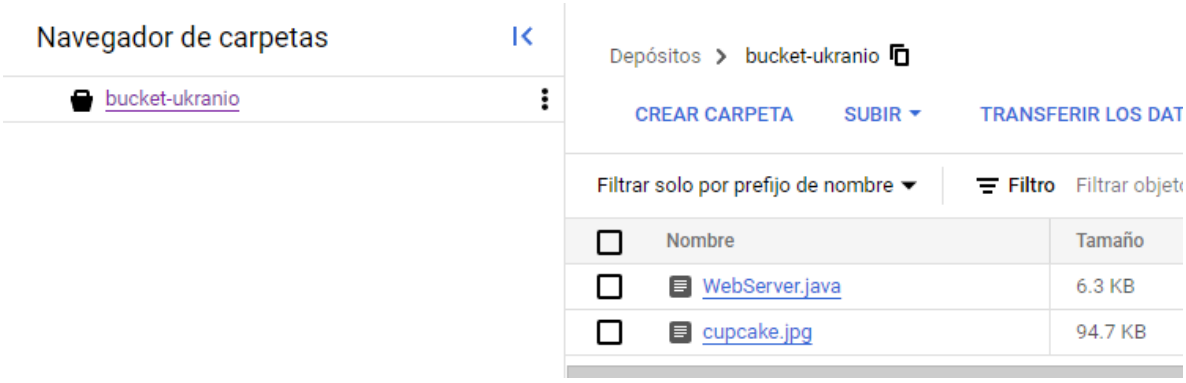
Google Cloud Storage JSON API

Elaborada por: M. en C. Ukranio Coronilla

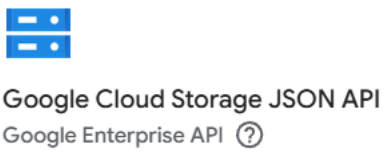
Los buckets son parte del servicio de almacenamiento de Google conocido como **Google Cloud Storage (GCS)**. Este además de la infraestructura de almacenamiento incluye APIs para acceder a los datos, y servicios de replicación entre otros.

En esta práctica mostraremos como se usa la **Google Cloud Storage JSON API**, la cual es una API RESTful que utiliza solicitudes HTTP para acceder y gestionar los datos en **GCS**.

Suba dos archivos (uno de código y otro de imagen) al bucket para usarlos de ejemplo dando click a Subir , con lo que en mi caso me ha quedado así:



Ahora proceda a habilitar el servicio de Google Cloud Storage JSON API:



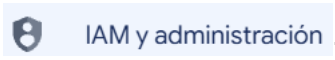
Dado que esta API implica acceso a datos sensibles y no sólo es un servicio de procesamiento como la **Traslation API**, requiere de un sistema más elaborado de credenciales y permisos de acceso.

Cabe aclarar que en Google Cloud Storage no existen en realidad archivos en carpetas, sino objetos identificados por una clave. Esto evita la jerarquía de archivos y permite el acceso directo a los objetos lo cual lo hace escalable y con mejor rendimiento distribuido.

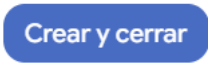
Identity and Access Management

Google Cloud dispone de un sistema para controlar quién puede acceder (usuarios, cuenta de servicio) a qué recursos (buckets, VMs, APIs) y qué acciones pueden realizarse sobre dichos objetos (crear, leer, escribir, borrar). Este sistema se denomina **Identity and Access Management (IAM)** y decide dar o no permiso después de que se haya dado una autenticación.

Para mostrar cómo funciona veremos como otorgar permisos sólo de lectura al bucket con los objetos que acabamos de subir, aunque lo podríamos hacer también sólo para un objeto dentro del bucket.

Primero creamos una **Cuenta de Servicio** para lo cual damos click a la hamburguesa ☰ y seleccionamos 

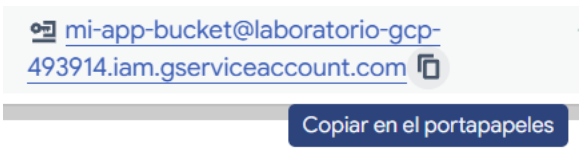
→**Cuentas de servicio**, posteriormente damos click en 

, le ponemos un nombre (en mi caso le puse `mi-app-bucket`) y le damos click en 

y nos crea la cuenta de servicio:



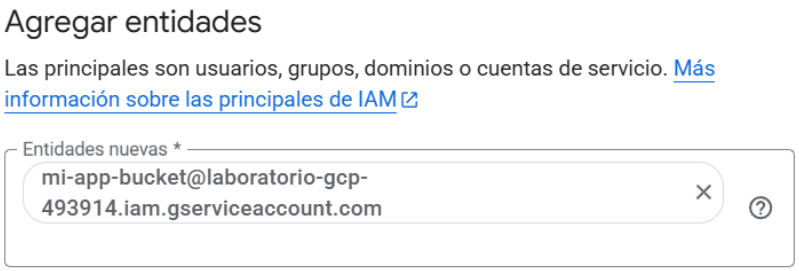
Observe que tiene asociado un correo electrónico, sin embargo, no es un correo donde se pueden enviar mails, se trata únicamente del nombre oficial de la cuenta de servicio dentro de la Google Cloud. A esta cuenta es a la que se la va a permitir leer un bucket o acceder a una API y funciona como referencia para que Google pueda auditarla en todo momento. Posicionamos el mouse sobre este correo para que aparezca el símbolo de copiado al cual le damos click:



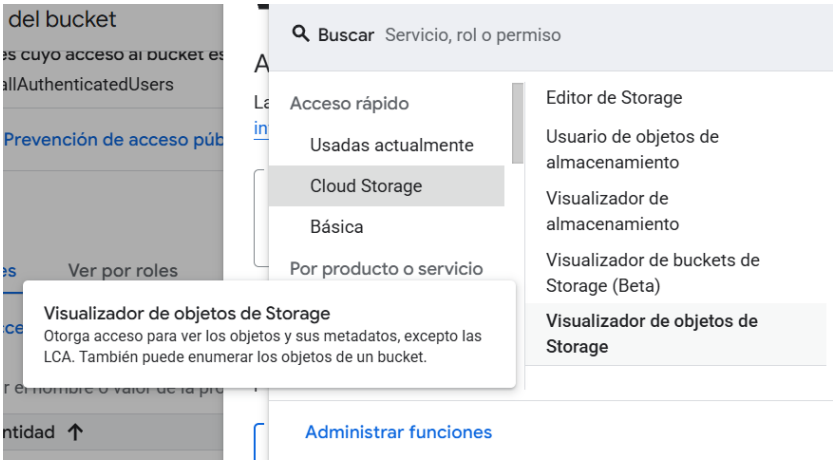
y pegamos dicho identificador en un editor de texto porque lo usaremos después.

Asignación de permisos al bucket

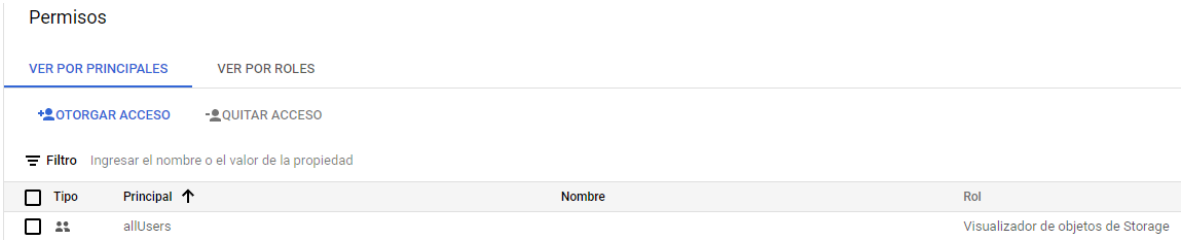
Ahora vamos a asociar nuestro bucket con la cuenta de servicio. En nuestro caso nos interesa asignar permisos al bucket así que le damos click a la hamburguesa ☰ seleccionamos: **Cloud Storage->Buckets** y le damos click a nuestro bucket. Ahora damos click a la pestaña Permisos y en la sección de Permisos damos click a + Otorgar acceso apareciendo una nueva ventana y en el recuadro de ***Entidades nuevas*** pegamos nuestra cuenta de servicio:



En la sección Asignar roles damos click en Seleccionar un rol y en el recuadro de la columna izquierda damos click en Cloud Storage:



En la columna derecha vienen diversos tipos de permisos, por ejemplo, si seleccionamos el permiso de ***Visualizador de objetos de Storage*** nos permite listar y descargar archivos, mientras que el permiso ***Administrador de objetos de Storage*** nos permite listar, subir y borrar archivos. En este caso como solo queremos acceder para lectura seleccionamos el permiso de ***Visualizador de objetos de Storage***. Finalmente damos click en Guardar tras lo cual podemos ver que se ha creado este nuevo permiso:



Si seleccionamos la casilla de verificación del permiso en la parte izquierda podremos quitar el permiso dando click en Quitar acceso si así lo deseamos.

Con este procedimiento le hemos dado permiso de lectura a los objetos que se encuentren en el bucket sólo a la cuenta de servicio que creamos.

Generación de credenciales JSON

En esta parte vamos a generar una credencial de acceso asociada con nuestra cuenta de servicio. Vamos a la cuenta de servicio que creamos con anterioridad y le damos click al correo electrónico, posteriormente le damos click a la pestaña Claves, luego damos click en Agregar clave y seleccionamos **Crear clave nueva**. En nuestro caso seleccionamos el tipo de clave JSON y automáticamente nos descarga el archivo de credenciales JSON.

Este archivo contiene la identificación de la cuenta de servicio junto con una contraseña, por lo que debe guardarlo en un sitio seguro, no se deja en el frontend y generalmente se almacena fuera del código fuente en el backend.

Generando manualmente un token de acceso

Lo que sigue es obtener un token de acceso temporal a la API de Google Cloud Storage (denominado **Access Token OAuth 2.0**), para lo cual accedemos al Cloud Shell y subimos el archivo de credenciales JSON dando click en ⋮ y después en Subir. Posteriormente nos autenticamos en la sesión con la cuenta de servicio que creamos ejecutando el comando (substituya en amarillo el nombre del archivo que subió y de preferencia en un editor antes de pegarlo en la consola):

```
gcloud auth activate-service-account --key-file=nombre_del_archivo.json
```

Ahora podemos obtener un token de acceso asociado a la cuenta de servicio con el siguiente comando:

```
gcloud auth print-access-token
```

Con lo cual nos devuelve un token de acceso que podemos usar para acceder al bucket. Copie dicho token y en la terminal UNIX de su LAP podrá descargar el recurso mediante curl como sigue (véanse los detalles de uso de la API en el manual):

```
curl -H "Authorization: Bearer token" "https://storage.googleapis.com/storage/v1/b/bucket-ukranio/o/cupcake.jpg?alt=media" -o cupcake.jpg
```

En azul va el token de acceso, en amarillo el nombre de su bucket y en verde el nombre del objeto dentro de su bucket (no se observa, pero existe un espacio en blanco entre las dos expresiones encerradas en comillas).

Cabe mencionar que el token de acceso tiene un periodo de vigencia de una hora a partir del momento en que se emite. En este caso el token se generó de manera manual, sin embargo, en la práctica este token se genera de manera automática en la interacción del cliente con la plataforma de Google, lo cual revisaremos en otro momento.

Consumiendo la API desde la aplicación

Para que nuestra aplicación en Java pueda descargar un archivo del bucket tiene que obtener primero el token OAuth 2.0 para poderlo insertar en el header `Authorization` del mensaje HTTP que va a solicitar el archivo, de la misma forma como lo hicimos con la petición con curl.

Obtener el token OAuth 2.0 conlleva varios pasos que se detallan a continuación:

1. Leer el archivo de credenciales JSON `credentials.json` para obtener `client_email` el cual es el correo de la cuenta de acceso, `private_key` que es una llave privada y la parte más sensible del archivo y finalmente `token_uri` que es el endpoint a donde hay que enviar la solicitud del token OAuth 2.0.
2. Se debe construir un JSON Web Token (JWT) el cual es un mensaje en formato JSON que contiene una solicitud firmada donde el algoritmo para firmarlo es RSA+SHA256. El JWT básicamente incluye el correo de la cuenta de acceso, lo que se desea hacer que en este caso es leer un archivo, y la firma equivalente a un hash aplicado a `private_key`.
3. Construido el JWT se envía al endpoint especificado en `token_uri` quien nos responde con el token OAuth 2.0.

El código que realiza estos tres pasos con ayuda de la librería Gson se muestra a continuación:

GoogleOAuthJWT.java

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.security.KeyFactory;
import java.security.PrivateKey;
import java.security.Signature;
import java.security.spec.PKCS8EncodedKeySpec;
import java.time.Instant;
import java.util.Base64;

import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

public class GoogleOAuthJWT {

    public static void main(String[] args) throws Exception {

        // 1. Leer JSON credentials
        String json = new String(Files.readAllBytes(Paths.get("credentials.json")));

        JsonObject creds = JsonParser.parseString(json).getAsJsonObject();

        String clientEmail = creds.get("client_email").getAsString();
        String privateKeyPem = creds.get("private_key").getAsString();
        String tokenUri = creds.get("token_uri").getAsString();

        // 2. Limpiar clave privada PEM
        privateKeyPem = privateKeyPem
                .replace("-----BEGIN PRIVATE KEY-----", "")
                .replace("-----END PRIVATE KEY-----", "")
                .replaceAll("\\s", "");

        byte[] keyBytes = Base64.getDecoder().decode(privateKeyPem);

        PKCS8EncodedKeySpec spec = new PKCS8EncodedKeySpec(keyBytes);
        KeyFactory kf = KeyFactory.getInstance("RSA");
        PrivateKey privateKey = kf.generatePrivate(spec);

        // 3. Crear Header JWT
        String headerJson = "{\"alg\":\"RS256\",\"typ\":\"JWT\"}";
        String header = base64UrlEncode(headerJson.getBytes(StandardCharsets.UTF_8));

        // 4. Crear Payload JWT
        long now = Instant.now().getEpochSecond();

        String payloadJson = "{"
                + "\"iss\":\"" + clientEmail + "\","
                + "\"scope\":\"https://www.googleapis.com/auth/devstorage.read_only\","
                + "\"aud\":\"" + tokenUri + "\","
                + "\"exp\":" + (now + 3600) + ","
                + "\"iat\":" + now
                + "}";

        String payload = base64UrlEncode(payloadJson.getBytes(StandardCharsets.UTF_8));

        // 5. Firmar JWT
        String unsignedToken = header + "." + payload;

        Signature signature = Signature.getInstance("SHA256withRSA");
        signature.initSign(privateKey);
        signature.update(unsignedToken.getBytes(StandardCharsets.UTF_8));
        byte[] signed = signature.sign();
```

```
        String jwt = unsignedToken + "." + base64UrlEncode(signed);

        // 6. Enviar a OAuth token endpoint
        String requestBody = "grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer"
                + "&assertion=" + jwt;

        HttpClient client = HttpClient.newHttpClient();

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(tokenUri))
                .header("Content-Type", "application/x-www-form-urlencoded")
                .POST(HttpRequest.BodyPublishers.ofString(requestBody))
                .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());

        // 7. Mostrar respuesta
        System.out.println("Respuesta:");
        System.out.println(response.body());
    }

    private static String base64UrlEncode(byte[] input) {
        return Base64.getUrlEncoder().withoutPadding().encodeToString(input);
    }
}
```

Ejercicio

Complete el código anterior para realizar la descarga de uno de los archivos que hay en el bucket. Envíe en un archivo pdf el código y la captura de pantalla de la consola UNIX donde se hace un listado largo de archivos, se ejecuta el programa y se vuelve a hacer el listado largo para mostrar que se ha descargado el archivo de manera exitosa.